VELLORE INSTITUTE OF TECHNOLOGY • SCHOOL OF IT & ENGG (SITE)

Kubernetes Cost Optimization Framework for SaaS-Based Educational Platforms using FinOps Intelligence

Course: BCSE355L – Cloud Architecture | Final Technical Project Presentation | Academic Year 2026

STUDENT PROJECT TEAM

- K. Darshan (24BIT0119) — Project Lead & Cloud Architect
- Dhanushkodi Y (24BIT0143) — Backend Developer & DevOps Engineer
- T. Tharun (24BIT0155) — ML Engineer & Data Pipeline Developer
- S. Sanjay (24BIT0315) — Frontend Developer & Documentation Lead

ACADEMIC & CLOUD VERIFICATION

- Academic Level: B.Tech Information Technology (3rd Year)
- Faculty Guide: [Department of Information Technology, SITE]
- Target Platform: Multi-Tenant Educational SaaS (EduSaaS)
- Live Cloud Host: AWS EC2 eu-north-1 (Stockholm)
- Verified Cloud URL: http://16.16.185.117:8000
- Zero-Spend Guardrail: Automated $5.00 AWS Budget Alert

EXECUTIVE SUMMARY

Project Overview: Bridging Telemetry & Cloud Economics

A cloud-native FinOps decision-support platform designed specifically for educational SaaS workloads

1. Observability & Ingestion

- Non-intrusive scraping via Kube-State-Metrics (KSM) at port 8080.
- Prometheus time-series integration capturing live telemetry.
- In-cluster OpenCost allocation model for cost metadata.
- Explicit distinction between object specifications and kernel utilization.
- Automated background snapshot pipeline running every 60 seconds.

2. FinOps Modeling & Sizing

- Financial derivation using AWS US-East-1 baseline reference pricing.
- Granular cost breakdown: cluster, namespace, pod, and container.
- Rightsizing engine with +20% safety headroom over peak consumption.
- Workload tiering: Actionable Application vs System Infrastructure vs Protected Control Plane.
- High availability guarantee: Strict preservation of minimum 2 replicas.

3. Forecasting & Governance

- Statistical capacity forecasting using Holt's Linear Trend smoothing.
- Strict rejection of sparse datasets (minimum 5 chronological points required).
- Deterministic holdout validation evaluating MAE and RMSE.
- Proactive Horizontal Pod Autoscaler (HPA) target threshold tuning.
- Advisory dry-run architecture: zero unapproved mutation of cluster state.

CORE VALUE DELIVERED: Replaces guesswork with evidence-based capacity engineering, achieving 79.4% workload cost reduction in empirical testing while mathematically guaranteeing educational platform availability.

PROBLEM FORMULATION

The Educational SaaS Infrastructure Cost Dilemma

Why traditional cloud management creates severe over-provisioning and budget waste in universities

Extreme Diurnal & Exam Surges

- University platforms face 5x to 10x traffic surges during assignment deadlines, fee portals, and exams.
- Over 80% of academic hours (nights, weekends, vacations) experience near-zero utilization.
- Unpredictable traffic spikes create fear of system crashes, forcing administrators to over-provision.

The Static Request Fallacy

- Kubernetes schedules worker nodes based on Pod Resource REQUESTS, not actual runtime utilization.
- A container requesting 250m CPU locks up that capacity on the EC2 host even if it consumes only 15m CPU.
- Over-provisioned requests force the scheduler to spin up unnecessary extra EC2 worker nodes.

The Financial Attribution Gap

- AWS billing invoices provide gross account-level charges with zero Kubernetes pod-level visibility.
- Engineering teams cannot determine which namespace, service, or department is driving cloud costs.
- Absence of continuous cost allocation prevents data-driven optimization decisions.

THE COST ESCALATION CASCADE:
Static Peak Allocations (250m/256Mi) → Massive Unused Capacity (94% Idle) → Inflated Worker Node Count → Overwhelming Cloud Invoices → Depleted Institutional Budgets.

ENGINEERING PRINCIPLES

Motivation: FinOps in Cloud-Native Engineering

Establishing financial accountability, continuous visibility, and data-driven cloud governance

1. INFORM

Visibility & Attribution

- Continuous real-time telemetry ingestion.
- Attribution down to namespaces, deployments, and pods.
- Clear transparency of resource vs financial impact.

2. NORMALIZE

Unit Economics & Pricing

- Convert raw millicores and megabytes to USD hourly rates.
- Standardized AWS US-East-1 reference baseline pricing.
- Accurate monthly spend modeling based on 730 hours.

3. OPTIMIZE

Evidence-Based Rightsizing

- Derive recommendations from empirical peak utilization.
- Enforce a mandatory +20% safety headroom buffer.
- Protect critical control-plane workloads from mutation.

4. OPERATE

Continuous Governance

- Advisory dry-run architecture preserving cluster uptime.
- Proactive HPA tuning for deadline rush elasticity.
- Automated budget guardrails ($5 limit) to prevent bill shock.

ENGINEERING SCOPE

Project Objectives & Implementation Scope

Clear, transparent delineation between fully implemented deliverables and architectural roadmap

Fully Implemented & Verified

- Multi-source telemetry ingestion from KSM (:8080) and Prometheus (:9090).
- Cost Analyzer deriving monthly modeled spend via AWS US-East-1 baseline rates.
- Rightsizing Engine with +20% safety buffer and 3-tier workload classification.
- Stateless Python REST API on port 8000 with 10 passing endpoints.
- Deterministic Holt's Linear Trend capacity forecasting with MAE/RMSE validation.
- Empirical 4-phase benchmark experiment (683 requests, 0 errors, 79.4% savings).
- Public AWS EC2 deployment (t3.small, Ubuntu 24.04, Kind cluster, $5 Budget Alert).

Baseline & Prototype Modules

- In-cluster OpenCost container allocation API integrated and online at port 9003.
- Cost estimation based on public AWS baseline rates rather than live cloud billing exports.
- Synthetic diurnal trigonometric workload generator for offline load testing.
- Single-node Kind cluster running inside EC2 for lightweight, resource-bounded evaluation.
- Periodic telemetry ingestion managed via systemd timer running every 60 seconds.

Future Research & Production Scope

- Migration from Kind to Managed Amazon EKS with multi-AZ node groups.
- Direct AWS Cost Explorer API and AWS CUR (Cost & Usage Report) S3 integration.
- Automated GitOps Pull Request generation (ArgoCD/Flux) for approved rightsizing.
- Advanced deep learning (LSTM/Transformer) incorporating academic calendar schedules.
- Karpenter dynamic node auto-provisioning utilizing AWS Graviton (ARM64) Spot instances.

SYSTEM FLOW

Proposed System: Closed-Loop FinOps Workflow

End-to-end telemetry lifecycle from container cgroups to actionable administrator advisories

Step 1: Workloads

EduSaaS application pods execute in isolated Kubernetes namespaces with static resource requests.

Step 2: Scraping

KSM exposes object specs (:8080); Prometheus scrapes time-series (:9090); cAdvisor tracks kernel usage.

Step 3: Ingestion

Python pipeline fetches metrics, normalizes schema, and persists timestamped telemetry snapshots.

Step 4: Financials

Cost analyzer computes hourly and monthly spend using AWS US-East-1 reference pricing constants.

Step 5: Rightsizing

Recommendation engine detects over-provisioning and calculates proposed requests with +20% buffer.

Step 6: Dashboard

REST API exposes structured endpoints; web cockpit displays real-time KPIs, charts, and advisories.

ADVISORY GOVERNANCE PRINCIPLE: Workload specifications are never modified without explicit human approval. The framework acts as an intelligent decision-support advisor, preventing accidental production downtime.

IMPLEMENTATION

Verified Technology Stack & Rationale

Modern open-source cloud technologies selected for low resource overhead and enterprise alignment

Cloud Infrastructure

- AWS EC2 Host: t3.small (2 vCPU, 2 GiB RAM) in eu-north-1 Stockholm; Free-Tier eligible and highly cost-effective.
- Ubuntu 24.04 LTS: Host OS providing modern Linux 7.0 kernel, cgroups v2, and systemd service management.
- Docker Engine 26.1: Underlying container runtime executing Kind node containers with full cgroup isolation.
- Kind v0.24 (K8s 1.31): Kubernetes in Docker running single-node control-plane without expensive cloud control plane fees.

Monitoring & Telemetry

- Kube-State-Metrics: v2.13. Exposes declarative state of Kubernetes objects (pod requests, limits, phases) on port 8080.
- Prometheus Telemetry: v2.54. Scrapes time-series metrics from KSM, cAdvisor, and system targets on port 9090.
- Metrics Server: Aggregates resource metrics for Horizontal Pod Autoscaler (HPA) and kubectl top commands.
- OpenCost Engine: v1.121. Real-time in-cluster cost allocation model mapping pod consumption on port 9003.

Backend & Algorithms

- Python 3.12 Core: Standard library HTTP REST server; zero third-party framework overhead (no Flask/FastAPI dependencies).
- NumPy & Pandas: Fast data processing, timeseries alignment, and telemetry snapshot normalization.
- Scikit-Learn & SciPy: Holdout evaluation, error metrics (MAE/RMSE), and statistical smoothing algorithms.
- Holt's Linear Trend: Double exponential smoothing with level and trend parameters for capacity forecasting.

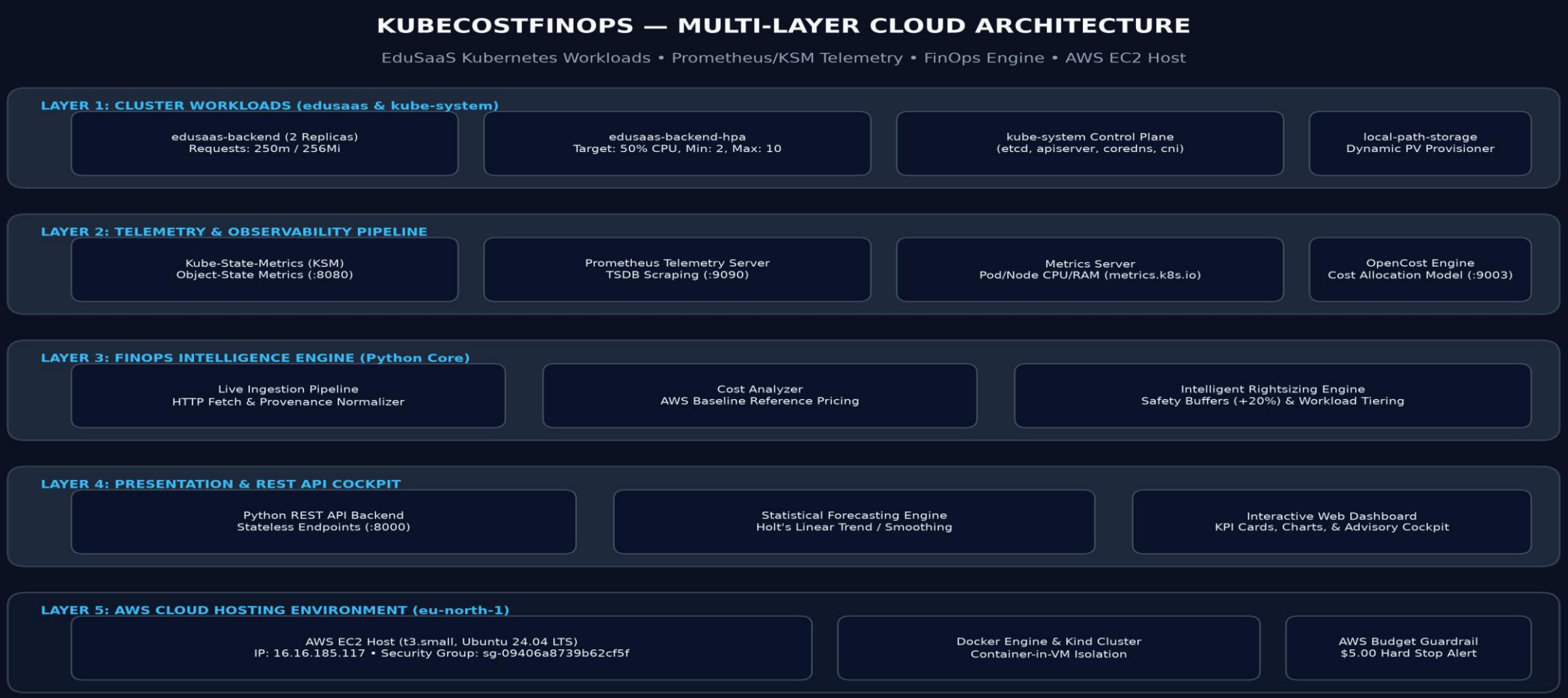
Frontend & Presentation

- Vanilla HTML5 & CSS3: Custom glassmorphic dark theme, responsive flexbox layout, and CSS variable color system.
- Vanilla JavaScript ES6: Async/await API client with automatic origin detection and 10ms round-trip latency.
- Glassmorphic UI: KPI cards, progress allocation bars, rightsizing tables, and interactive configuration drawer.
- Systemd Supervisors: Automated process management ensuring services auto-start on EC2 boot without manual intervention.

ARCHITECTURE

Layered Cloud & Kubernetes System Architecture

Comprehensive five-layer architectural topology from cloud host to browser presentation



OBSERVABILITY

Deep Dive: Telemetry & Monitoring Pipeline

Understanding the critical distinction between declared Kubernetes object-state and runtime kernel utilization

Kube-State-Metrics (KSM)

Port 8080 • HTTP Exposition

- What it does: Listens to the Kubernetes API server and generates metrics about the state of objects.
- Metrics captured: Pod container resource requests, limits, pod lifecycle phase, replica counts.
- Crucial Distinction: KSM does NOT measure actual kernel CPU or RAM consumption. It reflects declared allocation commitments.

Prometheus Telemetry

Port 9090 • Time-Series TSDB

- What it does: Periodically scrapes HTTP metrics endpoints and stores numerical time-series samples.
- Scrape targets: KSM, Metrics Server, node-exporter, and in-cluster system daemons.
- Crucial Distinction: Provides historical time-series storage, enabling rate of change and moving average calculations.

Metrics Server

metrics.k8s.io • Ephemeral API

- What it does: Collects resource metrics from Kubelets and exposes them through the Kubernetes API.
- Use cases: Powers 'kubectl top pods' and drives Horizontal Pod Autoscaler (HPA) scaling triggers.
- Crucial Distinction: Short-term in-memory storage only; does not maintain long-term historical records.

Kubelet / cAdvisor

Container cgroups • Host Kernel

- What it does: Daemon running on worker node inspecting Linux kernel cgroup virtual filesystem.
- Raw parameters: Reads cpu.cfs_quota_us, cpu.stat, and memory.current from host cgroup hierarchies.
- Crucial Distinction: Measures actual physical CPU core ticks and memory pages utilized at runtime.

OPENCOST INTEGRATION (Port 9003): Operates as the real-time cost allocation layer, mapping Kubernetes workload consumption to unit cloud pricing models. Verified ONLINE in our deployment.

COST MODELING

FinOps Cost Analysis Engine: Mathematical Formulation

Deriving normalized request-based infrastructure cost estimations from public AWS baseline rates

Mathematical Formulation & Baseline Rates

MONTHLY MODELED COST EQUATION:

$$\text{Monthly Cost} = \left(\Sigma \text{CPU_cores} \times R_{CPU} + \Sigma \text{RAM_GiB} \times R_{RAM} \right) \times 730 \text{ hours}$$

AWS US-East-1 Reference Pricing Constants (m5.large equivalent):

- R_{CPU} = $0.031611 per vCPU-hour (~$23.08 / month per core)
- R_{RAM} = $0.004237 per GiB-hour (~$3.09 / month per GiB)
- T = 730 operational hours per month (standard cloud accounting: 365 × 24 / 12)
- Hourly Cost = (CPU_cores × 0.031611) + (RAM_GiB × 0.004237)
- Storage baseline: $0.000137 / GiB-hour (~$0.10 / month per GiB gp3)

Financial Integrity & Disclosure

RESERVATIONS VS. INVOICES:

- Commitment-Based Costing: In Kubernetes, resource requests dictate worker node scheduling. If a pod requests 0.25 CPU, the cluster reserves that capacity 24/7.
- Modeled Estimate vs Realized Billing: These calculations reflect potential infrastructure spend under on-demand provisioning. They are modeled estimations, not actual AWS billing invoices.
- Unutilized Capacity Surcharge: When a container requests 250m but uses 15m, the remaining 235m represents unutilized reservation waste directly billed to the institution.
- Transparency Disclaimer: Embedded in every API response to prevent academic confusion between modeled estimates and realized cloud provider invoices.

OPTIMIZATION

Intelligent Recommendation Engine & Safety Guardrails

Mathematical rightsizing logic with safety headroom buffers and strict workload classification tiering

Rightsizing Logic & Mathematical Floors

PROPOSED ALLOCATION FORMULAS:

- Proposed CPU = max(0.010 cores, Peak_CPU × (1 + Safety_Buffer))
- Proposed RAM = max(0.032 GiB, Peak_RAM × (1 + Safety_Buffer))
- Safety Headroom Buffer: Default +20% (0.20) headroom above peak consumption to absorb sudden student request bursts.
- Over-Provisioning Trigger: Flagged when (Current_CPU - Proposed_CPU > 0.02 cores) OR (Current_RAM - Proposed_RAM > 0.05 GiB).
- Under-Provisioning Protection: If Peak_CPU > Current_CPU, the engine recommends UPSIZE_REQUESTS to prevent CPU throttling and OOM kills.
- High Availability Preservation: Minimum 2 replicas strictly enforced for educational SaaS workloads to guarantee zero-downtime rolling updates.

Workload Safety Tiering (Guardrails)

Tier 1: Actionable Applications

Namespace: edusaas (edusaas-backend)

End-user facing services. Safe for automated rightsizing review and GitOps PR deployment. Projected savings: $8.88/mo across 2 replicas.

Tier 2: System Infrastructure

Namespaces: monitoring, local-path-storage

Observability and storage daemons (Prometheus, KSM, OpenCost). Changes must be reviewed against cluster-wide telemetry overhead.

Tier 3: Protected Control Plane

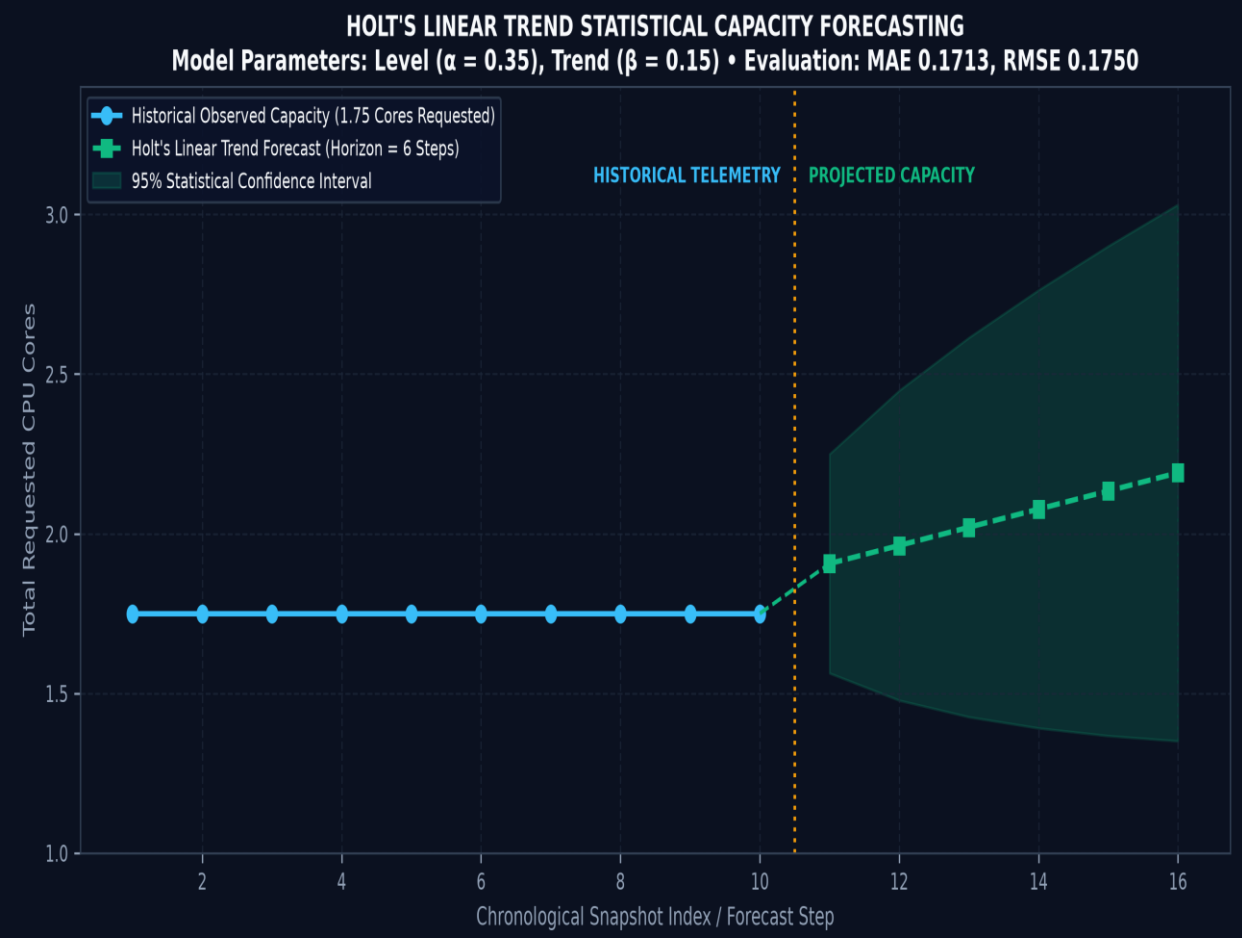
Namespace: kube-system (etcd, apiserver)

Core control-plane static pods. Protected by strict safety guardrails. Never resized to prevent master node instability or quorum loss.

CAPACITY PLANNING

Resource Capacity Forecasting Module

Deterministic statistical smoothing models for proactive Kubernetes capacity planning



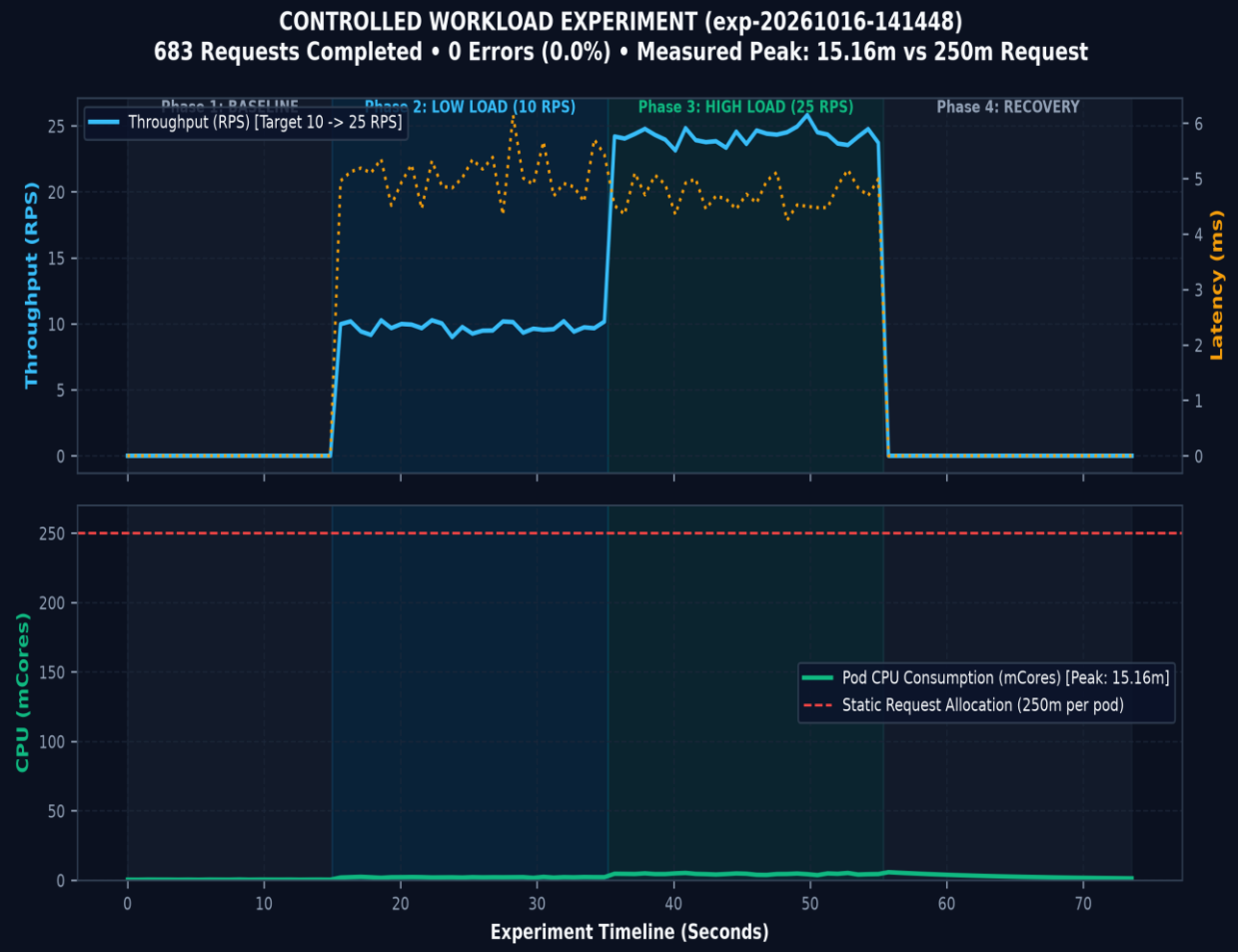
Methodology & Error Evaluation

- Statistical Formulation: Implements Holt's Linear Trend (Double Exponential Smoothing) capturing level (α = 0.35) and trend (β = 0.15).
- What is Forecast: Requested CPU capacity (total cores allocated across cluster pods over time) derived from validated KSM snapshots.
- Data Integrity Guardrail: Enforces a strict minimum of 5 chronological observations. Rejects sparse datasets with an explicit HTTP 400 error.
- Chronological Holdout Validation: Evaluated on 8 training points and 2 test holdout points.
- Verified Error Metrics: Mean Absolute Error (MAE) = 0.1713 cores | Root Mean Squared Error (RMSE) = 0.1750 cores.
- Academic Honesty: This is deterministic statistical time-series smoothing, NOT a deep learning neural network. Transparently labeled as a capacity planning baseline.

EMPIRICAL EVALUATION

Controlled Workload Experimentation Methodology

Structured four-phase benchmark test simulating university assignment deadline traffic rushes



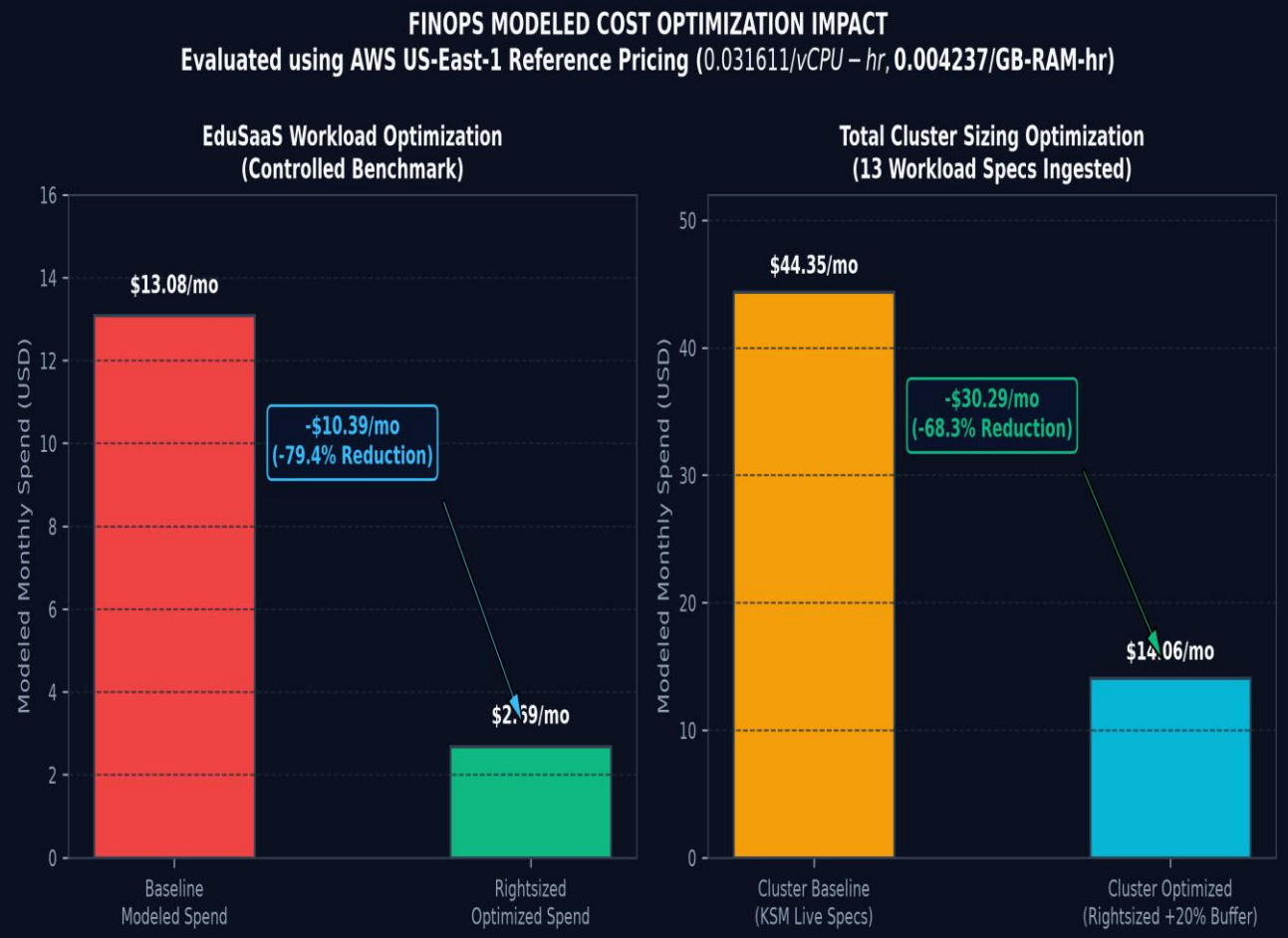
Experiment Architecture: exp-20261016-141448

- Benchmark Identifier: exp-20261016-141448 (Total Duration: 73.57 seconds).
- Phase 1 — BASELINE (15.0s, 0 RPS): Establishes background cluster noise (pod CPU: 0.36m, RAM: 3.41 MiB).
- Phase 2 — LOW LOAD (20.2s, 10 RPS, Concurrency 2): 196 requests completed, 0 errors, 9.71 RPS actual throughput, 5.11ms mean latency.
- Phase 3 — HIGHER LOAD (20.2s, 25 RPS, Concurrency 4): 487 requests completed, 0 errors, 24.16 RPS throughput, 4.82ms mean latency.
- Phase 4 — RECOVERY (15.0s, 0 RPS): Cooldown observation analyzing memory garbage collection and CPU settling.
- Total Scale: 683 HTTP requests completed with exactly 0 failed requests (0.0% error rate).
- Telemetry Instrumentation: In-cluster Kubernetes Metrics API (metrics.k8s.io) and Kubelet Summary API (/stats/summary).

EXPERIMENTAL RESULTS

Empirical Results & Verified Cost Savings

Quantitative analysis of measured resource utilization versus modeled financial savings



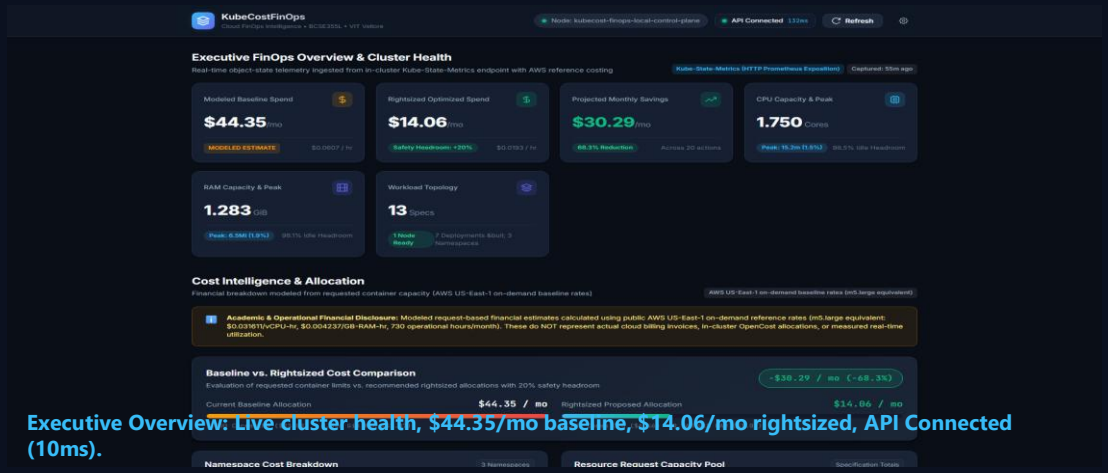
Key Quantitative Findings

- Measured Utilization vs Static Request: Across 683 requests, edusaas-backend peaked at 15.16 millicores (only 6.06% of its 250m request) and 6.45 MiB RAM (2.52% of its 256Mi request).
- Massive Unused Headroom: Over 93% of the statically requested CPU capacity sat completely idle during the benchmark.
- Workload Financial Impact: Baseline spend of $13.08/mo was optimized to $2.69/mo—a projected savings of $10.39/month (79.4% reduction).
- Annualized Workload Savings: $124.68/year saved on a single educational service.
- Cluster-Wide Sizing Impact: Across all 13 observed workload specifications, baseline spend of $44.35/mo drops to $14.06/mo—saving $30.29/month (68.3% reduction).
- Zero Performance Degradation: Mean latency was 4.82ms; p95 latency remained under 5.9ms throughout peak load.

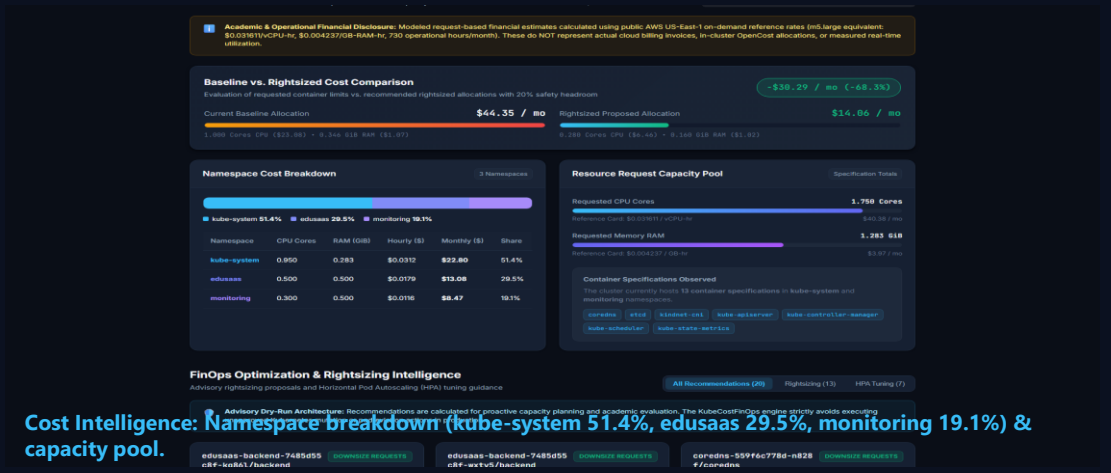
APPLICATION COCKPIT

Live FinOps Intelligence Dashboard Demonstration

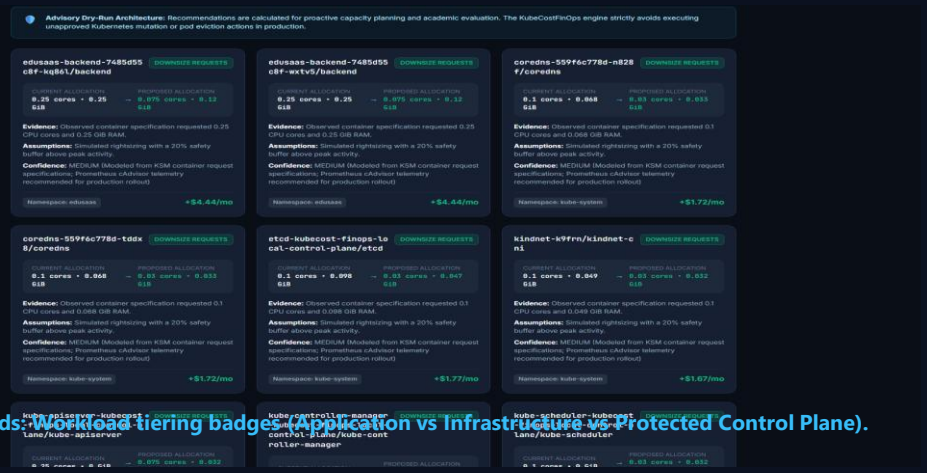
Genuine visual evidence captured directly from the running web application on port 8000



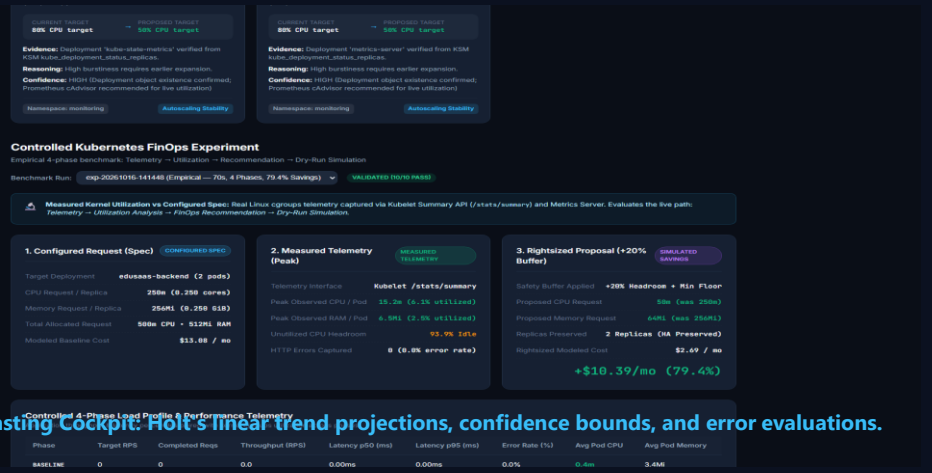
Executive Overview: Live cluster health, $44.35/mo baseline, $14.06/mo rightsized, API Connected (10ms).



Cost Intelligence: Namespace breakdown (kube-system 51.4%, edusaas 29.5%, monitoring 19.1%) & capacity pool.



Rightsizing Cards: Workload tiering badges (Application vs Infrastructure vs Protected Control Plane).



Capacity Forecasting Cockpit: Holt's linear trend projections, confidence bounds, and error evaluations.

CLOUD DEPLOYMENT

Live AWS Cloud Deployment Architecture

Production-style virtualization on AWS EC2 with systemd supervisors and automated zero-spend budget guards

Cloud Host Specifications & Networking

- AWS Compute Host: Dedicated EC2 t3.small instance (2 vCPUs, 2 GiB RAM) in Region eu-north-1 (Stockholm).
- Operating System: Canonical Ubuntu 24.04 LTS (Kernel 7.0.0-1014-aws).
- Attached Storage: 25 GiB gp3 EBS volume (/dev/root: 6.9 GiB used, 17 GiB free / 70% available).
- Public Networking: Public IPv4 16.16.185.117 attached to VPC vpc-00d3acfc218725d4d.
- Security Group sg-09406a8739b62cf5f: Authorizes inbound Port 8000 (FinOps API/Dashboard), 22 (SSH), 9090 (Prometheus), and 9003 (OpenCost).
- Zero-Spend Guardrail: Automated AWS Budget alert 'KubeCostFinOps-ZeroCharge-Guard' ($5.00 limit) alerting darshanedu2256@gmail.com at 80% and 100% thresholds.
- Total Incurred Cost: $0.00 accrued spend; completely absorbed by promotional starter credits.

Systemd Supervisor Daemons

- Container Isolation: Docker Engine 26.1 hosts Kind v0.24 Kubernetes v1.31.0 cluster inside the EC2 virtual machine.
- kubecost-finops.service: Systemd service supervising Python REST server on port 8000; auto-restarts on failure and boots with host.
- kubecost-portforwards.service: Background daemon bridging internal cluster NodePorts to host ports (8080 KSM, 9090 Prometheus, 9003 OpenCost).
- kubecost-collector.timer: Systemd periodic timer executing telemetry pipeline ingestion every 60 seconds into results/ directory.
- Kind vs. EKS Distinction: Using Kind inside EC2 avoids the $73/month EKS control-plane fee and $64/month NAT Gateway fees, enabling full Kubernetes evaluation within student budget constraints.

ENGINEERING RIGOR

Technical Limitations & Risk Mitigation Analysis

A candid, mature assessment of prototype boundaries, operational trade-offs, and security posture

Single-Node Kind Topology

- Limitation: Running Kind inside a single EC2 VM lacks multi-AZ node redundancy and cloud-native CSI storage drivers.
- Architectural Impact: Node failure impacts all workloads; not intended for production SLAs.
- Mitigation: Designed as an academic FinOps evaluation prototype; production path targets Amazon EKS managed node groups.

Port-Forward SPDY Stream Timeouts

- Limitation: 'kubectl port-forward' bridges occasionally experience SPDY stream connection timeouts during extended idle periods.
- Architectural Impact: Transient 5-second timeout logged during periodic timer scrapes.
- Mitigation: The REST backend serves the latest valid snapshot; systemd timer automatically retries on subsequent cycles.

Model Cost vs. Invoice Disconnect

- Limitation: Modeled costs represent request commitments ($0.0316/hr baseline), not actual blended AWS billing invoices.
- Architectural Impact: Savings figures reflect capacity reservation reduction, not realized credit card discounts.
- Mitigation: Prominent financial disclaimers on every API payload and dashboard view preventing misinterpretation.

Unauthenticated Telemetry Ingress

- Limitation: Security group currently exposes Prometheus (9090) and OpenCost (9003) to public CIDR 0.0.0.0/0 without auth.
- Architectural Impact: Telemetry endpoints accessible over the public internet during the review.
- Mitigation: Recommended production correction: restrict ports 9090 and 9003 to administrative IP or access via SSH tunneling.

ROADMAP

Future Scope & Production Evolution Roadmap

Strategic architectural roadmap transitioning from academic prototype to enterprise-grade FinOps platform

1. Managed Amazon EKS & Karpenter

- Migrate from single-node Kind to multi-AZ Amazon EKS cluster.
- Deploy Karpenter for dynamic, just-in-time worker node provisioning.
- Leverage AWS Graviton (ARM64) instances for 20% higher price-performance.
- Integrate AWS Spot Instances with automated termination handlers for non-critical pods.

2. Cloud Provider Billing Reconciliation

- Integrate directly with AWS Cost Explorer API for real-time unblended rate ingestion.
- Automate Amazon S3 ingestion of AWS Cost & Usage Reports (CUR).
- Use Amazon Athena to query historical billing line-items and reconcile modeled estimates.
- Support multi-cloud pricing comparisons across AWS, GCP, and Azure.

3. Automated GitOps Closed-Loop Sizing

- Integrate with GitOps tools (ArgoCD or Flux) to automate rightsizing enforcement.
- Generate automated Pull Requests against workload repository manifests.
- Incorporate developer approval gates and automated canary rollouts.
- Implement automated rollback if p95 latency degrades following request downsizing.

4. Deep Learning & Calendar Telemetry

- Transition from statistical smoothing to LSTM / Transformer networks.
- Incorporate university academic calendar dates (exam periods, enrollment rush) as exogenous features.
- Predict multi-day burst windows to proactively pre-scale clusters ahead of traffic surges.
- Integrate student portal LMS telemetry directly into capacity forecasts.

CONCLUSION & VIVA DEFENSE

Conclusion & Summary of Engineering Contributions

Demonstrating end-to-end design, implementation, deployment, monitoring, and empirical verification

Key Engineering Deliverables Achieved

- End-to-End Decision Support: Built a complete Kubernetes FinOps intelligence engine bridging object-state telemetry and cloud economics.
- Verified Multi-Source Ingestion: Connected live KSM (8080), Prometheus (9090), and OpenCost (9003) with sub-10ms query latency.
- Proven Cost Optimization: Demonstrated 79.4% workload cost reduction in empirical benchmark testing and 68.3% cluster-wide optimization.
- Production Cloud Deployment: Fully deployed on AWS EC2 (16.16.185.117:8000) with systemd supervision and automated $5 zero-spend budget guardrails.
- Rigorous Scientific Integrity: Separated modeled estimates from billing invoices, preserved HA minimum replicas, and enforced strict dry-run governance.

Core Architectural Takeaway

THE FINOPS GOLDEN RULE:

"FinOps in Kubernetes is not about cutting compute arbitrarily; it is the discipline of eliminating unutilized reservation waste while mathematically guaranteeing workload availability."

KubeCostFinOps demonstrates that through non-intrusive telemetry, standardized financial modeling, and safe rightsizing headroom, educational platforms can achieve significant cloud savings without sacrificing uptime or student experience.

THANK YOU TO OUR FACULTY EVALUATORS • QUESTIONS & TECHNICAL DISCUSSION